

Applying Agile Through Total Immersion

Going All-In with Agile at PH Tech

Chad Casady

Director of Information Technology
Performance Health Technology
Salem, Oregon, USA
E-mail: chad.casady@phtech.com

Abstract – *This paper describes a multi-year journey of a growing small business and the transition from the blissful ignorance of cowboy-coding through the misery and despair of a sinking, dysfunctional organization, and on to the satisfaction and exhilaration that comes with a high-functioning agile team. This writer wishes that you, dear reader, learns from the mistakes in sections II through IV, and starts their team at section V.*

Keywords– *extreme programming; agile; software development; team development; agile immersion*

I. INTRODUCTION

Implementing agile in a poorly functioning team is commonly done using one of three techniques, which include piecemeal, prolonged, and immersive implementation. In the piecemeal method, an organization will select specific agile practices to use; usually ones that a manager or lead employee believes will address an area that needs the most improvement. Some organizations use the prolonged method, where the agile team is developed and specific agile practices are introduced over a long period of time -- often two years or more. In the immersion method, an agile coach is employed to build a new team from the very start within an environment that utilizes agile methods, or to “reboot” an existing team by throwing out all existing development practices and tools and replacing them with agile methods.

Through anecdotal conversations with agile consultants and other “agilists,” it is my understanding that many organizations attempt to repair a poorly functioning team by using either a combination of the piecemeal and prolonged methods of agile implementation. While ultimately improvements are often made, the results of this method are seldom as effective and valuable as a complete immersion guided by an experienced agile coach. Through the years, many competing software development practices were tried and discarded as Performance Health Technology (PH Tech) used a Do It Yourself (DIY) approach to software development.

By using this DIY approach, dysfunction grew at PH Tech throughout the software development lifecycle over a period of ten years, and finally culminated in an environment where five individuals of the software engineering department more often worked *against* each other than *with* each other, causing productivity to be scarce.

II. COWBOY CODING BLISS

As the software team was formed in the early stages, there were relatively few standard software development practices utilized, aside from the use of source control and basic coding, database, and architecture standards which were understood among the team members. The software team consisted of two or three developers who wrote code independently of each other, and deployed it to production systems on an as-needed basis. Occasionally, code was written or modified directly on production systems in order to respond quickly to customer requests or troubleshoot problems.

During this period, developer satisfaction was easily achieved by responding immediately to customer requests. The team was able to handle requests quickly, albeit with little or no testing, and with frequent mistakes. The team was able to move quickly and shift easily to develop new tools that became necessary. However, there was no particular long-range product or architectural planning that occurred, and as demand for software enhancements and bug fixes increased, it became apparent that the development “practices” as they existed were unsustainable and needed to be changed.

III. COMMITTEE-BASED PROCESS IMPROVEMENT

As it is with many organizations and teams who do not have a thorough understanding of how to efficiently and effectively build software, PH Tech arrived at the conclusion that quality would be improved by implementing bureaucratic processes. For several years, PH Tech experienced the pain associated with the addition of multiple steps in the software lifecycle process, under the false assumption that by increasing the number of times an individual or group “signs-off” or

“approves” a step in the process, the quality of the final product will ultimately be higher.

One specific step during the software development process that a majority of organizations implement is that of Quality Assurance (QA). In some software companies, it is believed that the best way to ensure that a software product meets quality standards is to have a step in the development process in which a dedicated person or team of people works to find faults in the software. In general, the QA process works in this way: when a fault in the software is found by the quality assurance team, the software is rejected and returned to the development team who must fix the problem, and then re-submit it to the QA team for another review. More often than not, the more faults that are found, the more likely the QA team is perceived to be effective and the software development team to be error-prone.

PH Tech management saw other organizations employ this method, and thus a Quality Assurance Engineer was hired to fill the role of assuring software quality. Far too many years later, it became more obvious that this did little more than to address poor quality by focusing on the symptom, rather than the cause.

After the quality assurance step was implemented, PH Tech began to experiment with several versions of a process to determine the enhancements, bugs, and new features that were important enough to use software engineering team resources. The first version of this approval process consisted of a committee made up of the Software Engineering Manager, Operational Managers, and the Leadership Team, and whose purpose was to evaluate and prioritize all software requests. The Software Engineering Manager’s role in this committee was to provide estimates of the number of hours required for each step in the development process (architecture, coding, testing and quality assurance, and documentation), and to make recommendations as to which items could be done easily as well as which items were somewhat ambiguous and would require more research.

This Technical Steering Committee had many meetings that often resulted in re-prioritizing the entire list, which involved moving the pet projects or what was considered low-hanging fruit to the top of the list regardless of their value to the organization or simply saying yes to everything with little consideration to the resources required. Rarely, if ever, was anything removed from the list. Many times a project would be moved to the top of the priority list ahead of something which was already in-progress. This primarily caused two things to happen. The first was a reset of the development team as they jumped from one project to the next and the second was

frustration among the developers. This resulted in reduced productivity that in turn caused the list to grow and more re-prioritizing to occur – giving birth to a vicious circle.

During the period in which PH Tech was pursuing this particular process improvement method, both management and staff became increasingly frustrated with the output of the Software Engineering department. Between poor software quality and low productivity, the team was often unable to meet the expectations of the rest of the organization, and became further disconnected from the users of the software that they were building.

Management became concerned that developers were not spending enough time writing code, and reasoned that it was necessary to implement a time-tracking application in order to log the amount of time spent in meetings, on projects, and at lunch. In hindsight, it’s now clear that this was the obvious, critical key indicator that distrust was reaching a feverish peak and continued to spread rampantly throughout the organization. The software engineering department became collectively dissatisfied with their jobs, and increasingly turned against each other through workplace sabotage and personal attacks.

IV. THE TROUBLES

The saying, “The beatings will continue until morale improves” represents something that seemed so ridiculous no one could possibly believe, that is, until I actually lived it. In order to bring order to chaos, improve the process, and deliver high quality software, I hired a new Software Engineering Manager to run the department. This new manager had experience in several other similar-sized software development teams and was committed to the waterfall method for building software. Hiring this manager was the worst mistake of my professional career, as well as the most educational. This manager’s methods for improving the team included:

- Using email to berate the team for deploying poor quality software while carbon-copying the Leadership Team,
- Informing development staff of the new team philosophy of “you’ll get enough rope to hang yourself,”
- Implementing a rigid waterfall method heavily dependent upon gathering detailed requirements (down to the specific size of the database field for a data element) and then getting customer sign-off of those requirements before the first line of code is written.

After six months, we concluded that this manager was leading our Software Engineering department in the wrong direction, and I made the decision to terminate. At

the time, this appeared to be rock bottom, and it was close. Unfortunately, we had more to learn.

For a short time, after the removal of the new manager and a reduction in beatings, morale did actually improve, and several projects were completed with renewed enthusiasm. However, the mendacious euphoria was short-lived, and because we had still not eliminated any of the original issues experienced prior to my hiring of the manager, the team sank back into deeper dysfunction, and ultimately into what can only be described as depression.

Distrust and blame was everywhere. The team didn't trust each other. Leadership didn't trust the team. The team didn't trust me. Leadership didn't trust me. The team didn't trust the leadership. *I didn't trust myself.* I had made the egregious mistake of attempting to fix the problem by hiring the wrong manager, and the situation was such that PH Tech's owners were evaluating whether it was even reasonable to maintain a software engineering department. Two particular dire options that were realistically being considered included terminating the entire software engineering department and outsource all development, or buying a competing software package instead of developing our own. Both options would have a dramatic impact across the entire organization. The fact that they were even being considered made me feel as though I was personally failing both the team and the entire company.

In April 2009, while attending the Innotech Conference in Portland, I selected a session that included a panel of speakers discussing agile development practices. It was 45 minutes that changed everything. One of the speakers who particularly impressed me was an agile consultant named James "Jim" Shore of Portland, Oregon, who authored *The Art of Agile Development* [1]. At the end of the session, I emailed his web site URL to myself for additional research when I returned from the conference. After a few phone calls and meetings with him, I was ready to make a proposal to the CEO to implement agile at PH Tech.

V. GOING ALL-IN

After much enthusiastic debate, the Leadership Team made the decision that having a functional software engineering team is one of PH Tech's strategic advantages, and investing in it was an investment in the entire organization. This decision made it easier to swallow the cost of hiring a consultant and committing to the agile immersion. Even so, there was still a significant amount of pessimism disguised as realism throughout the organization as the agile immersion began.

In June 2009, PH Tech began its agile immersion with the help of Jim Shore as an experienced Agile Coach.

Much of his expertise is in the transformation of software development teams into agile teams.

Before the agile immersion could begin, Jim required that the Leadership Team of PH Tech commit to supporting and contributing to the immersion in a positive way. This was really nothing more than a signature on a piece of paper, but it was a key factor in the success of the transformation. Jim made it clear to the top management that this process would be painful, and that there will be moments during the transition when not everyone will be satisfied. In order for the immersion to be successful, management needed to understand that their participation and support was vital throughout these times of dissatisfaction, and that they would have the opportunity on a weekly basis to choose to continue or abandon the process.

Before starting, each member of the Software Engineering department was also required to make an individual commitment to the agile transition. What was intriguing during this process was that no matter how much pain and dysfunction these employees were experiencing every day in their work life, they still found it difficult to commit to making this change. They understood that this would completely change how they worked and that it could improve it significantly, but even though they were so incredibly dissatisfied with the current situation, the *fear* of change was almost overwhelming. This insight has been one of the most powerful things I've learned in my career. Do not underestimate the fear of change.

VI. TOTAL IMMERSION

The transition began by wiping the slate clean. All software development processes, workflows, and steps that were currently happening were evaluated to understand whether they were delivering any value, and were either refurbished, or removed entirely. Everything, including the existing source control system, was subject to eradication. In one exercise, key staff members participated in value-stream mapping the software development life-cycle, and it was brought to light that many of the additional bureaucracies added during the Committee-Based Process Improvement era actually contributed to a majority of the 18-month software cycle time. Those bureaucracies were among the first processes to be eliminated.

The agile transition was structured around the following key principles for Extreme Programming (XP), not coincidentally matching the chapters of Jim's book [1] titled Thinking, Collaborating, Releasing, Planning, and Developing. These principles are the foundation of every agile practice that the team implements.

The team began with a crash course in how to change the way they solve problems. Prior to this, members of the team worked in their own office or cubicle and for the most part, solved problems on an individual basis. In an XP team, pair programming is used frequently if not 100% of the time, and members of the team continually work with each other to perform their daily work. Root-cause analysis was also an important focus, as it encouraged the team to blame the process rather than each other when problems appeared. After each weekly iteration, the team performed a retrospective in order to understand how to continually improve work habits using the same root-cause analysis to solve team problems as they did technical problems.

Collaboration among the team wasn't nearly as difficult to get started as was collaboration with customers. Having customers (internal users are considered customers) sitting with the Software Engineering team while they were writing code was a new experience for everyone. Getting time commitment from those customers to sit with the engineers was a struggle at first, but it eventually became less time intensive for the customer as the team developed domain expertise themselves.

Releasing code was a significant hurdle for quite some time as the team learned and implemented XP practices. Before this, the process by which software was deployed to production servers was convoluted at best and haphazard at worst. It was exceptionally difficult for the team to build and deploy code on a weekly cadence since it could take weeks to deploy the changes to production. Resolving this was exceptionally important in order to enable the team to move quickly and respond to customer requests. Even today, this continues to be a primary factor in enabling the team to continue to increase their velocity.

Project and Product Planning was essentially done by committee prior to the agile transition. There was no particular owner of these functions so there were a significant number of disconnected pieces and inconsistent designs within PH Tech's applications. To change that, the team selected one member who would primarily perform the role of Product Manager for the team and organization. The Product Manager is responsible for fielding and prioritizing requests from customers, researching and understanding the connectivity of those requests with other parts of the application and users, as well as rejecting requests that the team cannot accept due to resource constraints.

Test-Driven Development (TDD) was a very difficult concept for the team to grasp. Writing tests prior to writing any code was a foreign concept – for that matter,

before this time, there were very few tests written by developers *after* writing code, so tests in general were somewhat of a foreign concept. It took a few months for TDD to become comfortable, but once it became part of the development routine, it provided a significant amount of value.

The idea of refactoring was a popular one, as there are a large number of legacy applications at PH Tech that are supported by the Software Engineering team. The team began the refactoring process on a very small scale – one or two lines of code at a time – and eventually was able to scale that to refactoring larger pieces of the application with confidence.

After just two weeks into the transition, the software engineering team was deep into the group development stage of forming [2], and was beginning to experience the benefit, albeit unpleasantness, of storming. The team was beginning to understand the new processes and techniques, how to work within the new system, and how to work with each other, possibly for the first time.

Every Thursday all employees in the organization were invited to attend a weekly demo presented by the software engineering team. These demos gave the opportunity for the team to reconnect with the rest of the organization and to improve transparency into what the team was working on. It was after these weekly demos that the team met with PH Tech's owners and asked these two key questions:

- 1) "Are you satisfied with the results this week?"
- 2) "May we continue?"

The questions were the same each week, and enabled two very important things to happen. The first was direct and immediate feedback from top management to the software engineering team, and the second was public re-assertion of support for the agile transition process by top management. It was also a subtle way of communicating that any dissatisfaction expressed outside of this forum would undermine the chance of success – honest and straightforward feedback was to become the new norm.

By the end of the second month, the team was consistently improving and delivering software on a weekly cadence and had established its pace. By the end of the fourth month, the team was able to function almost entirely without daily guidance from Jim and trust within the team was beginning to take shape across the organization. Clear progress was being made, and the level of productivity was visible and transparent to all. By the end of six months and the immersion program, the team was stable and ready to move into the next stage of the transition.

While the Software Engineering team was learning agile methods and implementing new practices through immersion, additional coaching was necessary for me as the team's manager, in order to learn how to develop and work with a self-organizing team on an on-going basis. PH Tech hired Diana Larsen of FutureWorks Consulting, and while the Software Engineering team was learning new development methods, I was learning how to support and enable them in new ways. Until this time, PH Tech had only known traditional management methods, which made building a self-organizing team at the heart of the company a new experience for everyone, and outside assistance was critical to learn how to overcome the challenges and transition to agile management.

VII. WINNING

At the beginning of 2010, PH Tech contracted Arlo Belshee as an Agile Coach in order to further develop and expand the software engineering team's technical skills and to help them learn how to evaluate and refine agile practices for themselves. By this time, the team was comfortable with who they were, the job they were there to do, and how to work with each other. They had survived the most dramatic part of the agile immersion, and were now ready to refine the skills they had learned.

During 2010, PH Tech was also able to hire four additional developers. This was a significant achievement because these were the first new hires for this department since late 2005 – nearly five years earlier. Until this time, there was little confidence by management that adding a developer would contribute to productivity in a reasonable amount of time or that there would be a return on investment.

By August 2010, the software engineering team was beginning to feel constrained by the Agile Coach, and was ready to continue working without one. At least two members of the team at this time communicated in earnest, "please don't renew his contract!" To some team members at the time, it seemed as though the contracted coach was no longer performing to expectations, when in fact he had achieved exactly what he was hired to do – help the software engineering team complete the steps toward becoming self-organizing.

Today, using an "Impact Trend" chart as seen in Figure 1, the software team is able to look back and analyze the impact of specific events on velocity. Using this information, PH Tech is able to more accurately predict what will happen with the team's productivity when specific types of events occur, such as when engineers leave or join the team, or when an external disruption occurs.

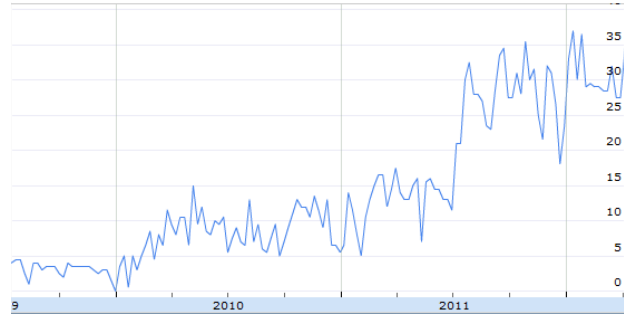


Figure 1: Impact Trend

VIII. GIVING BACK

The path PH Tech travelled to develop a self-organizing agile software development team has become the most painful and rewarding experience in my professional career. Here are a few observations and learned lessons that may provide additional insights for others in the agile (or non-agile) community:

- Do not underestimate fear of change – people instinctually fear what they do not know or understand. The way to overcome this is to educate them and provide them with the opportunity to make the choice to participate or not.
- Passive support is not support – in this case, it was a necessity that management and the leadership of the organization actively participate and support the transition to agile. The team needed to see that the company was behind them and that they were not doing it alone.
- A true agile team manager is not a manager – it is exceptionally important that managers of an agile team receive training or coaching to help them understand their role within the team. The golden rule for an agile team manager is "Remove obstacles whenever necessary and make sure you're never one of them."
- Finding the communication balance is important – not enough communication causes distrust, and too much communication can cause either disengagement or over-reaction. In our organization, communication was a weakness between all parties involved, so finding the right balance was exceptionally important.
- Agile is not for everyone – it is less likely that introverts will thrive in an agile environment, particularly one that practices Extreme Programming (XP). It's critical both for team

members and management to be aware and sensitive of this.

ACKNOWLEDGMENT

To Jim Shore, who I feel like I should still be paying though his contract ended 2 years ago.

To Diana Larsen, who wisely advised me to stay the heck out of their way.

To Dr. Michael Rohwer, PH Tech's CEO, who said "Go for it" when I suggested we go all-in.

To Nate Perrizo, PH Tech's Director of Operations, for his continued support and coaching during The Troubles and beyond.

REFERENCES

- [1] J. Shore and S. Warden, "The Art of Agile Development," O'Reilly Media, 2007.
- [2] B. Tuckman, "Developmental Sequence In Small Groups," Psychological Bulletin 1965 Jun; 63:384–99.